

Lezione 7: Threads & Concorrenza



Obiettivi

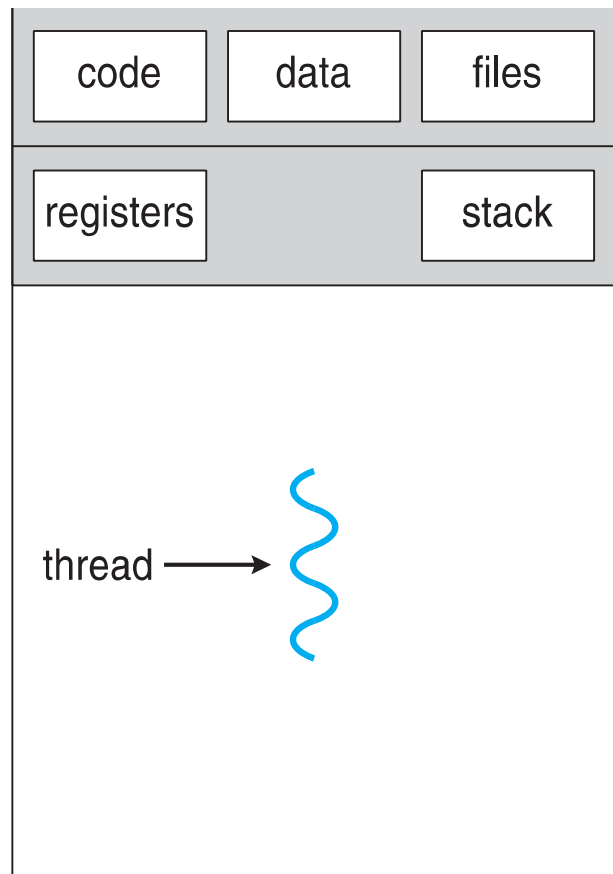
- Introdurre la nozione di thread
- Multithreaded programming
- Supporti per threads in Windows e Linux

Motivazioni

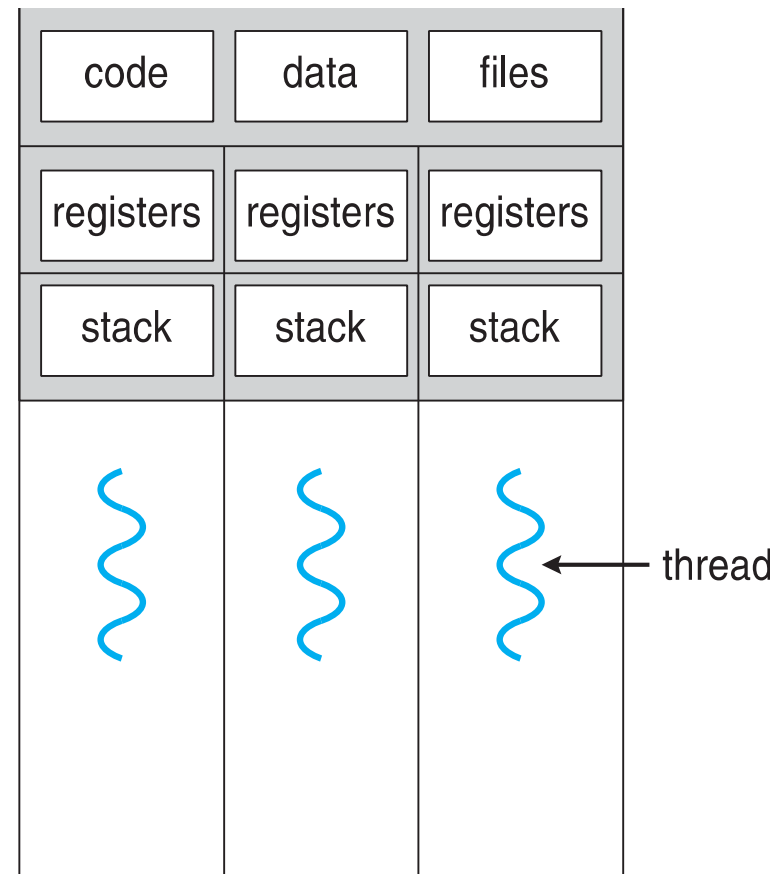
- ❑ Le applicazioni sono generalmente multithreaded
- ❑ Task multipli in applicazioni implementati da thread separati
 - ❑ Update del display
 - ❑ Fetch data
 - ❑ Spell checking
 - ❑ Risposta in rete
- ❑ La creazione di processo è pesante, il thread è leggero
- ❑ Il kernel è generalmente multithreaded

Processi Single e Multithreaded

- Unità base di utilizzo di CPU
 - Caratterizzati da Thread ID, PC, Registri e Stack
 - Minima identità di task



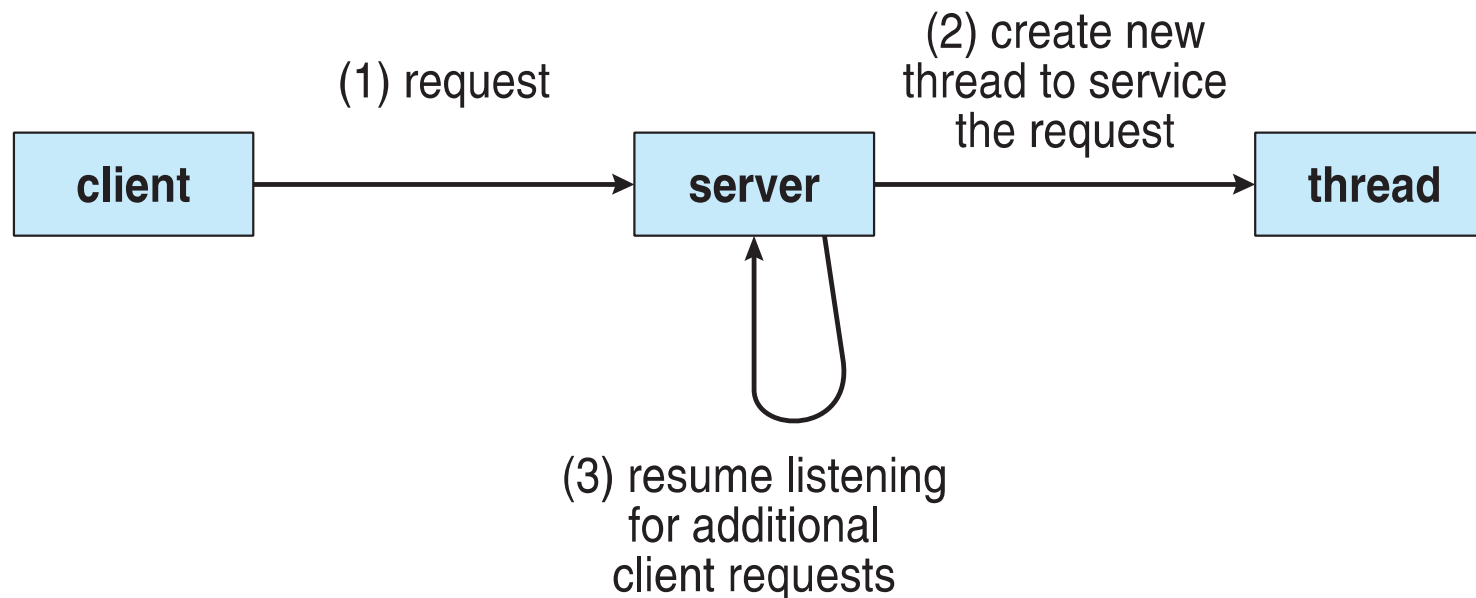
single-threaded process



multithreaded process

Architettura di Server Multithread

- Molto utilizzate nelle architetture client-server
 - Per ogni client si genera un thread che gestisce il client
 - Per ogni richiesta di un client un thread
 - Server può gestire tante richieste eseguendo molti task leggeri



Benefici

□ Risposta

- Esecuzione continua se una parte del processo è bloccata (importante per le interfacce utente)

□ Risorse

- Thread condividono le risorse di un processo, più semplice di shared memory o message passing

□ Economia

- Più leggero di una creazione di processo, il **thread switching** ha minore overhead del **context switching**

□ Scalabilità

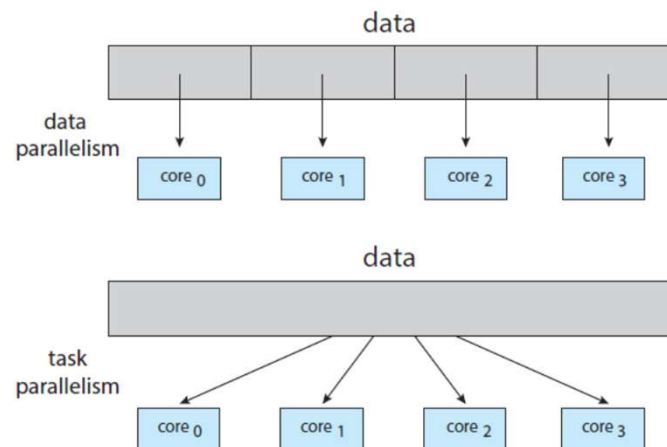
- I processi possono avvantaggiarsi delle architetture multiprocessore

Multicore Programming

- ❑ Sistemi multitread forniscono meccanismi utili per sistemi multicore
- ❑ Sistemi **Multicore** o **multiprocessori** pongono diversi problemi:
 - ❑ **Come dividere le attività sui core**
 - ▶ Computazioni parallelizzabili su più core
 - ❑ **Come bilanciare i calcoli sui core**
 - ❑ **Come splittare i dati**
 - ❑ **Come gestire le dipendenze dei dati sui core**
 - ▶ Dipendenze possono richiedere sincronizzazioni
 - ❑ **Come fare test e debugging**
 - ▶ Più tracce di esecuzioni da testare
- ❑ **Parallelismo**
 - ❑ Più di un task simultaneamente distribuiti su più unità di calcolo
- ❑ **Concorrenza**
 - ❑ Consente il progresso contemporaneo di più di un task
 - ❑ Se processore/core singolo, concorrenza data dallo scheduler che intrallaccia le esecuzioni

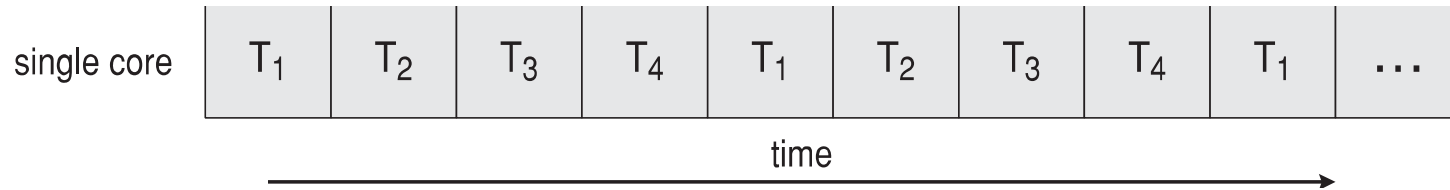
Multicore Programming

- Tipi di parallelismo
 - **Parallelismo di dati** – sottoinsiemi dei dati su core multipli con stessa operazione per ognuno
 - ▶ Ad esempio calcolo della somma dei numeri su array, si può dividere il compito
 - **Parallelismo di task** – thread sui core dove ciascuno svolge un'operazione particolare
 - ▶ Esempio media e varianza di un set di dati
- Al crescere del numero dei thread aumenta il supporto architetturale
 - CPU hanno core e **hardware thread**
 - Esempio, i7 (13th Gen) con 16 core e 24 hardware threads per core

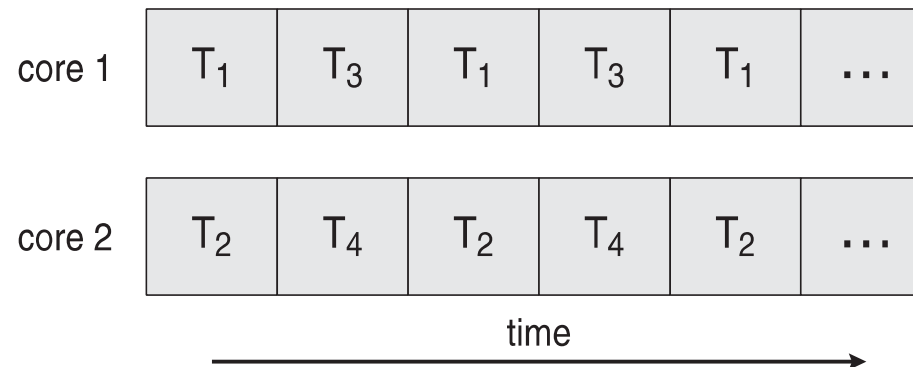


Concorrenza vs. Parallelismo

□ Esecuzione concorrente su sistema single-core:



□ Parallelismo su sistema multi-core:



Legge di Amdahl

- Guadagno teorico in performance con l'aggiunta di un core per un'applicazione che ha sia componenti seriali che paralleli
 - S è la porzione seriale
 - N sono i core per il processamento

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

speed-up rispetto ad un singolo core

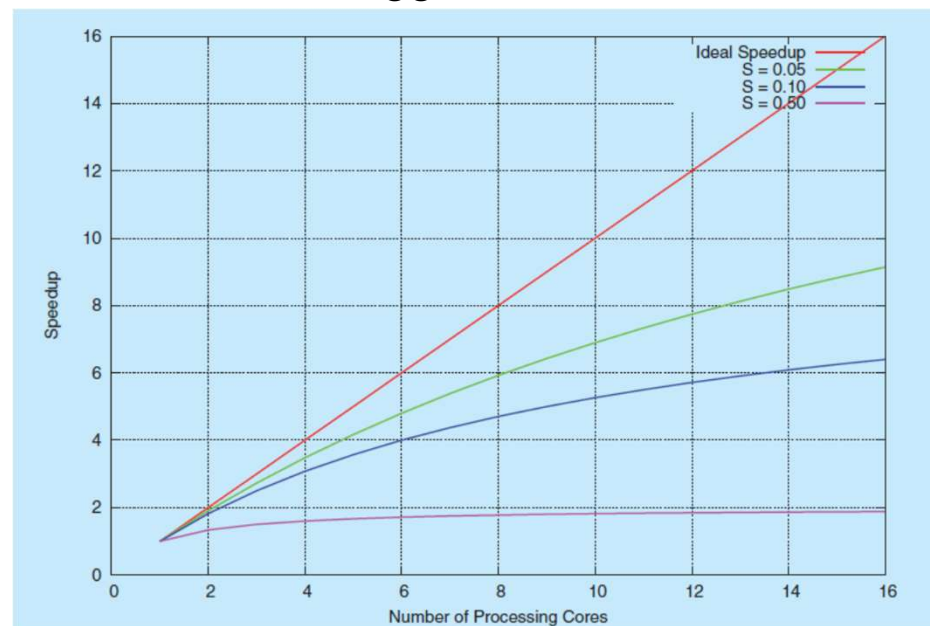
- Esempio: se l'applicazione è al 75% parallela e 25% seriale, passando da 1 a 2 core si ha uno speed-up di 1.6 volte
- Con N che tende ad infinito lo speed-up tende a $1 / S$

Legge di Amdahl

- Guadagno teorico in performance (latenza) con l'aggiunta di un core per un'applicazione che ha sia componenti seriali che paralleli
 - S è la porzione di calcolo seriale (non parallelizzabile)
 - N sono i core per il processamento

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- La porzione seriale di un'applicazione ha effetto molto marcato sul guadagno di performance con un core aggiuntivo



User Thread e Kernel Thread

- **User thread** – gestita da librerie di thread di livello utente

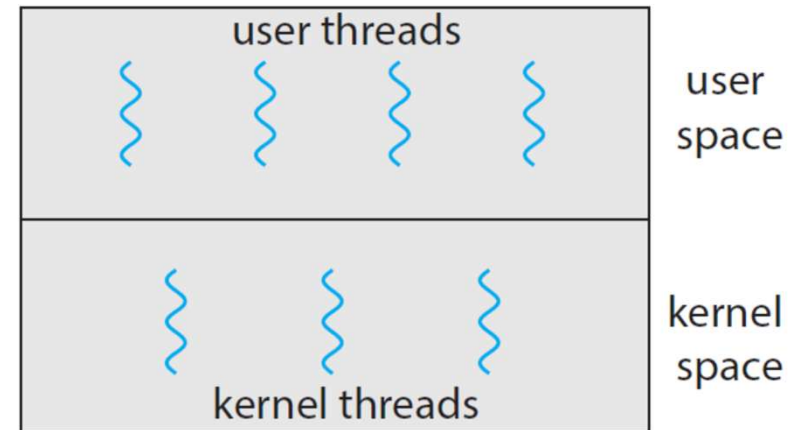
- Tre principali:

- POSIX **Pthreads**
- Windows threads
- Java threads

- **Kernel thread** - supportati dal Kernel

- Esempi – tutti gli SO general purpose inclusi:

- Windows
- Solaris
- Linux
- Tru64 UNIX
- Mac OS

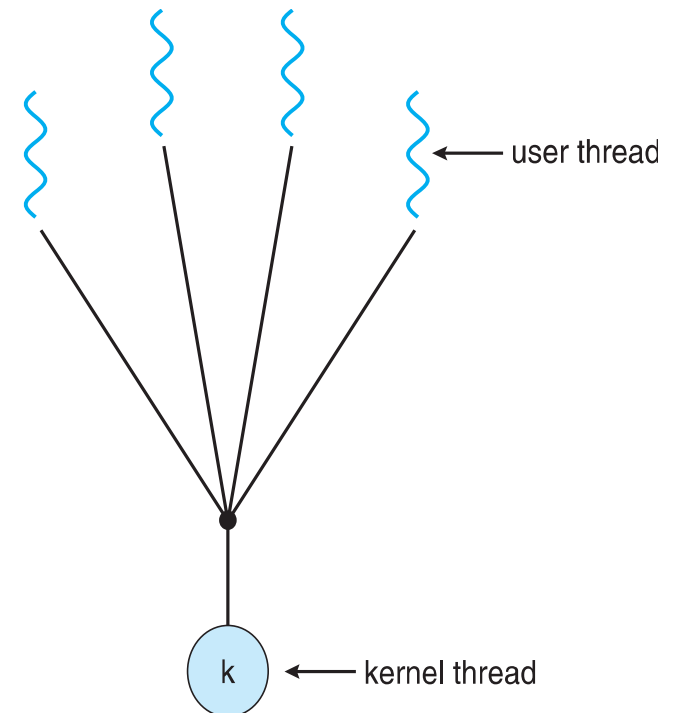


Modelli di Multithreading

- Diversi modi di gestire la relazione tra User e Kernel thread
 - Many-to-One
 - One-to-One
 - Many-to-Many

Many-to-One

- ❑ Tanti thread user mappati su un singolo kernel thread
- ❑ Però un thread bloccante (es. durante una system call) può bloccare tutto
- ❑ Problemi di parallelismo con sistemi multicore perché uno solo alla volta è in esecuzione nel Kernel
- ❑ Pochi sistemi utilizzano questo approccio:
 - ❑ **Solaris Green Threads** library
 - ❑ **GNU Portable Threads** library



Many-to-One

- Tanti thread user mappati su un singolo kernel thread
 - Lato kernel un solo PCB per gestire i thread di un processo

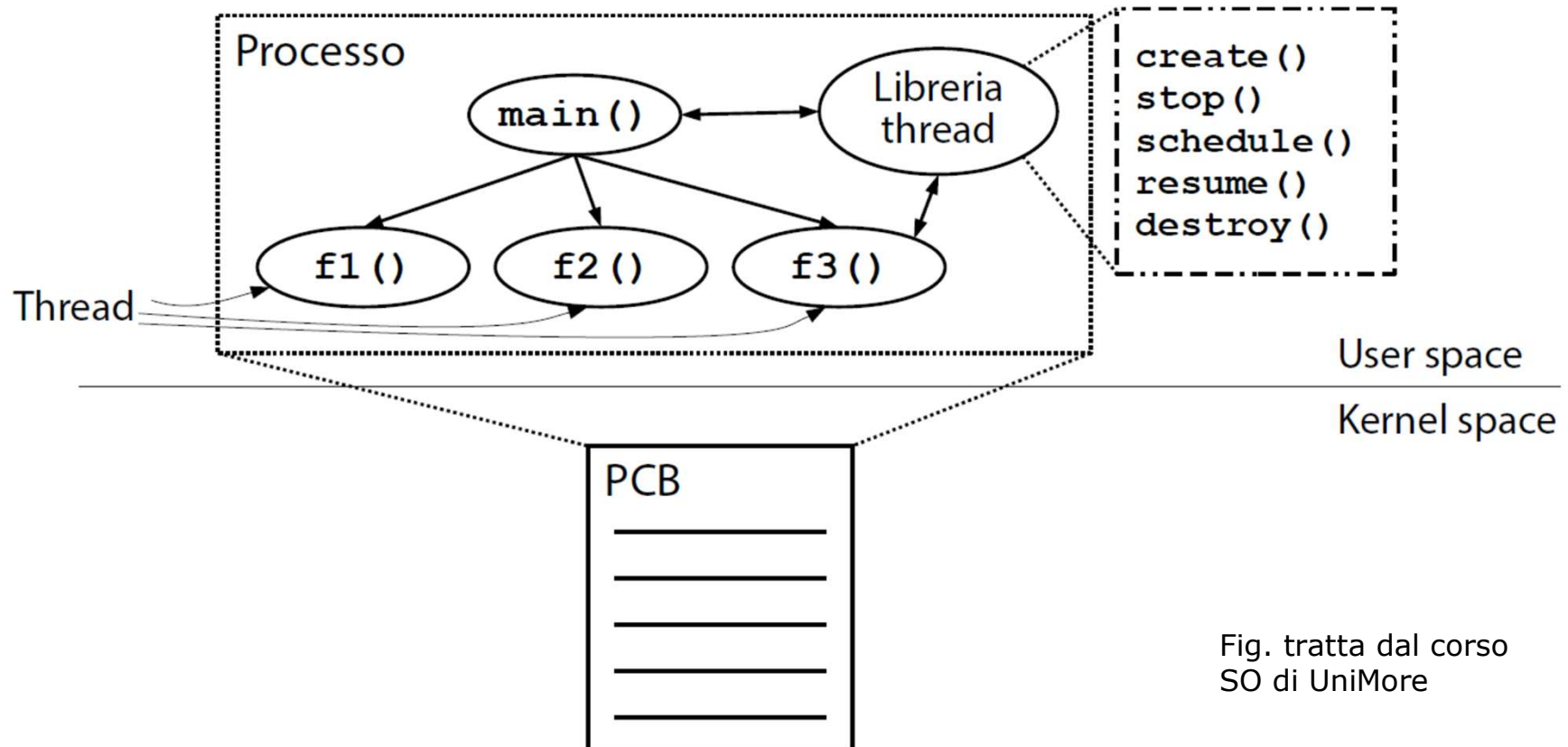
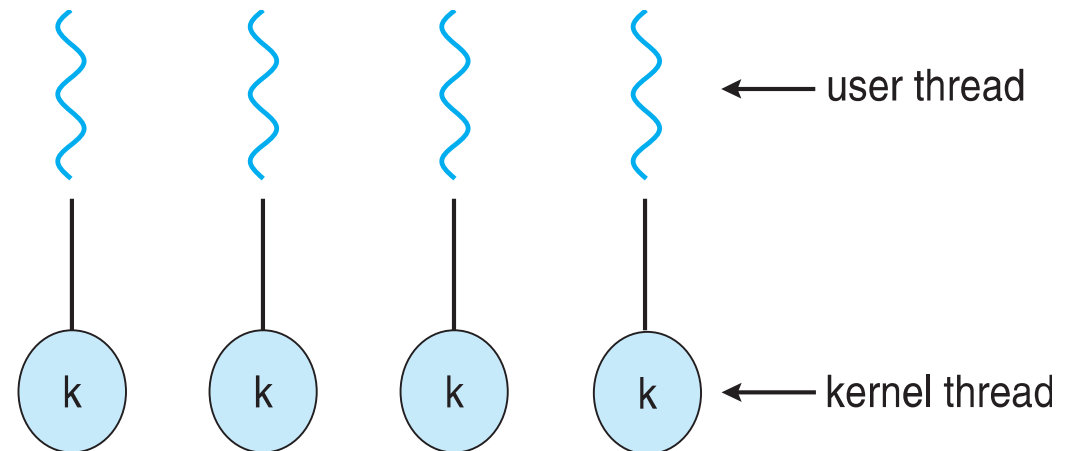


Fig. tratta dal corso
SO di UniMore

One-to-One

- Ogni thread user-level mappato su kernel thread
 - La creazione di thread user-level crea un kernel thread
 - Maggiore concorrenza di many-to-one
 - Il numero di thread per processo può essere limitato per evitare overhead
 - Esempi:
 - ▶ Windows
 - ▶ Linux
 - ▶ Solaris 9 e successivi
- Utente deve controllare il numero dei thread



One-to-One

- Ogni thread user-level mappato su kernel thread
 - La creazione di thread user-level crea un kernel thread

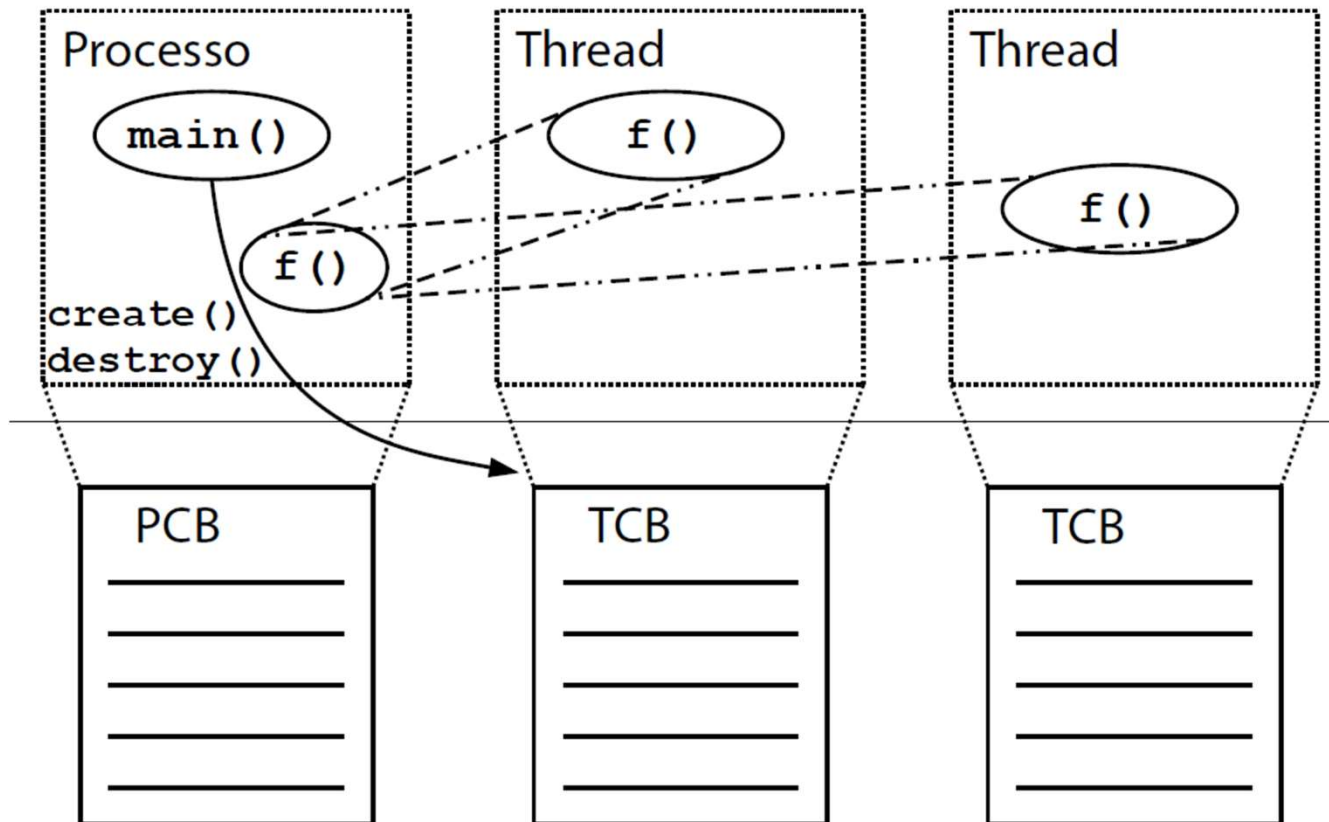


Fig. tratta dal corso
SO di UniMore

Many-to-Many

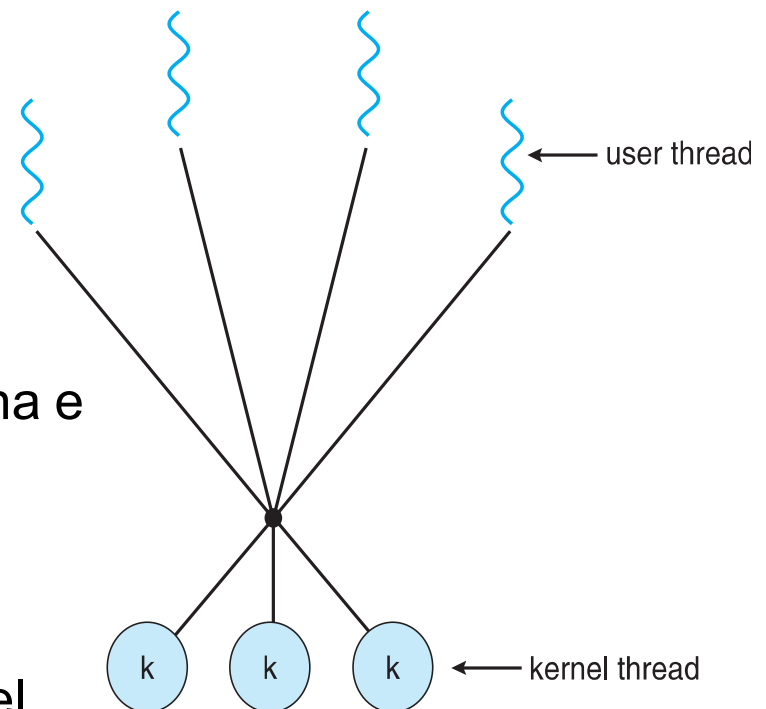
- ❑ Molti thread user-level sono mappati su molti thread del kernel
- ❑ Il SO può creare un numero sufficiente di thread del kernel
 - ❑ Però può gestirne il numero e il grado di parallelismo

- ❑ Esempi:

- ❑ Solaris dalla versione 9
 - ❑ Windows con *ThreadFiber* package

- ❑ Il numero può dipendere dalla macchina e dall'applicazione

- ❑ L'utente può lanciare più thread, poi controlla il Sistema quanti thread kernel



Many-to-Many

- ❑ Molti thread user-level sono mappati su molti thread del kernel
- ❑ Il SO può creare un numero sufficiente di thread del kernel
 - ❑ Però può gestirne il numero e il grado di parallelismo

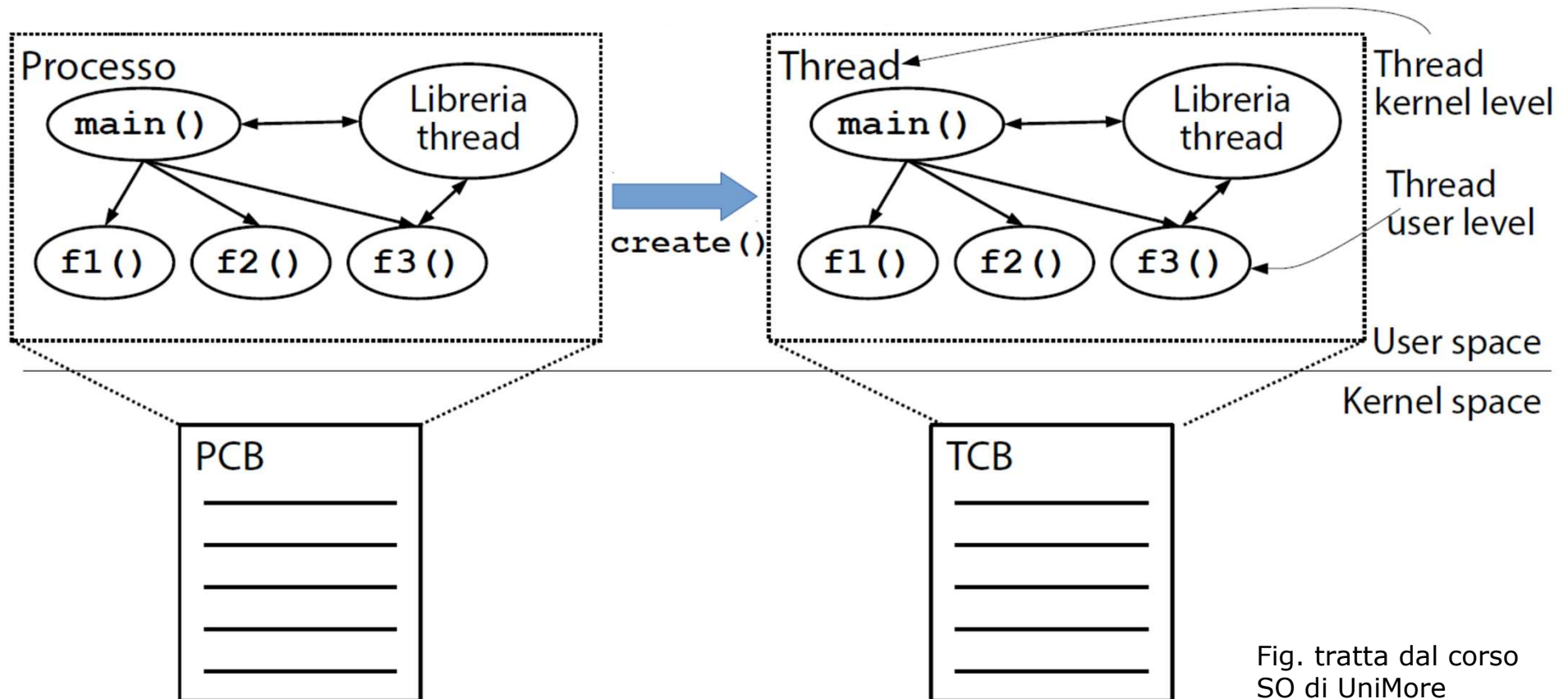


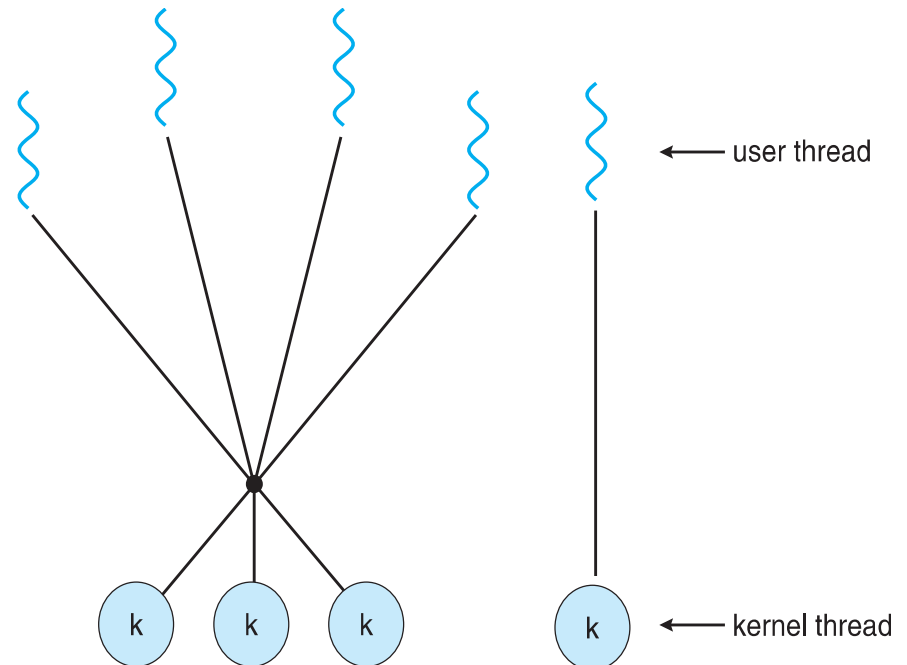
Fig. tratta dal corso
SO di UniMore

Two-level Model

- Variante del Many-to-Many che permette ad un user-thread di essere legato (**bound**) ad un kernel thread

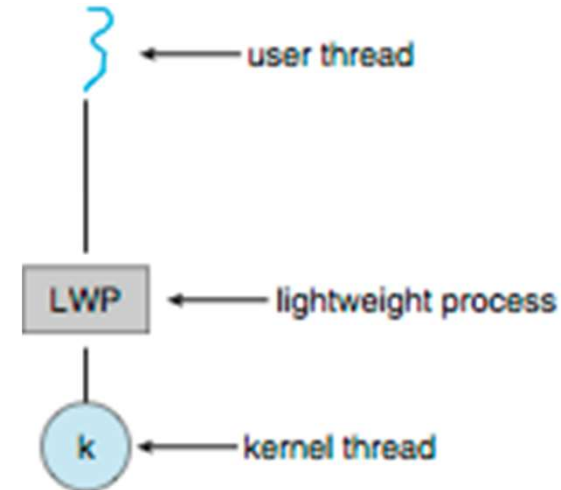
- Esempi:

- IRIX
- HP-UX
- Tru64 UNIX
- Solaris 8 e precedenti



Attivazioni Scheduler

- Sia modelli Many-to-Many che Two-level richiedono di mantenere un numero appropriato di kernel thread allocati per l'applicazione
- Tipicamente usata una struttura dati intermedia tra user e kernel thread – **lightweight process (LWP)**
 - Come un processore virtuale su cui il processo può schedare l'esecuzione di user thread
 - Ogni LWP è associato ad un kernel thread
- Il kernel fornisce ad un'applicazione un insieme di LWPs e l'applicazione può schedare gli user thread su questi
- Il kernel deve comunicare con l'applicazione
 - **upcalls** – quando si sta per bloccare il kernel è gestito da **upcall-handler**
 - ▶ comunica all'applicazione che sta per bloccarsi crea un nuovo LWP
 - ▶ consente all'applicazione di schedare un nuovo processo
 - Questa comunicazione consente ad un'applicazione di mantenere il corretto numero di kernel thread



Librerie per Thread

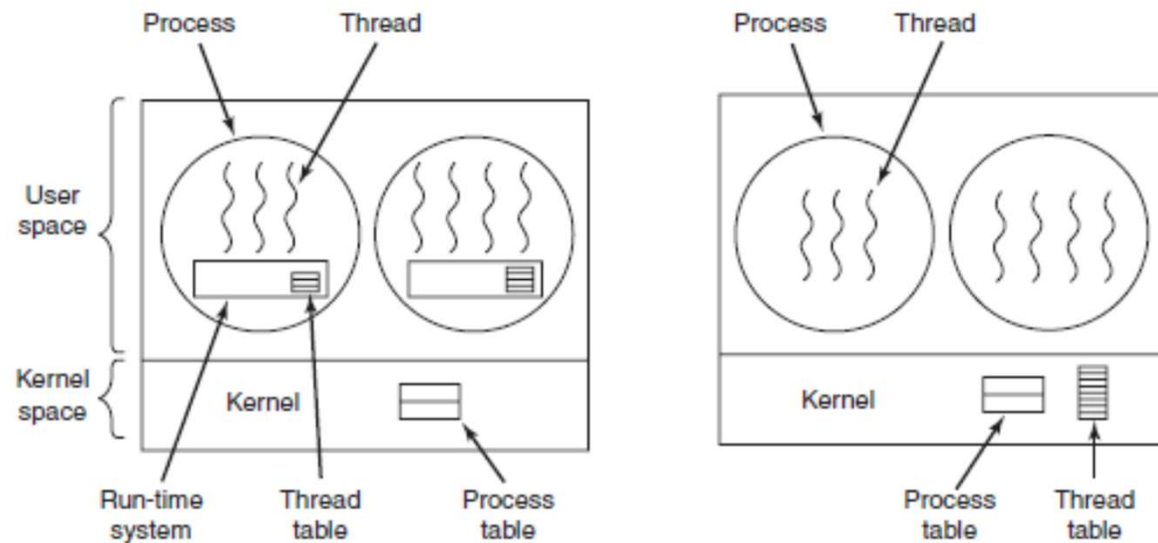
- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread

- Due modi principali di implementarle
 - Libreria interamente nello user space
 - ▶ I thread sono implementati al livello utente (gestiti con chiamate di funzione utente)

 - Libreria al livello Kernel supportata dal SO
 - ▶ I thread sono gestiti con chiamate di Sistema (gestiti con chiamate di sistema)

Librerie per Thread

- **Librerie per i thread (Thread library)**
 - API per creare e gestire i thread
- Due modi principali di implementarle
 - Libreria interamente nello user space
 - Libreria al livello Kernel supportata dall'SO



Pthreads

- Estensione di POSIX standard (IEEE 1003.1c) fornisce le API per la creazione e sincronizzazione dei thread
- Esiste sia in versione user-level sia kernel-level
- **Specifica**, non **implementazione**
- API specifica il comportamento delle funzioni di libreria, implementazione lasciata allo sviluppatore
- Tipiche in UNIX (Solaris, Linux, Mac OS)

Esempio Pthreads

- Esempio somma dei primi N numeri
 - Per sempio per $N = 5$, $\text{Sum} = 1 + 2 + 3 + 4 + 5 = 15$
 - Si può delegare il calcolo ad uno o più thread
 - Abbiamo thread **sincroni** e **asincroni**
 - ▶ Nel caso sincrono il genitore aspetta i figli
 - ▶ Nel caso asincrono il genitore continua subito

Esempio Pthreads

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```

Esempio Pthreads

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Codice Pthreads per il join di 10 Thread

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Pthreads

Per creare thread addizionali relativi ad uno stesso processo, Posix prevede la funzione:

```
#include <pthread.h>
```

```
int pthread_create ( pthread_t *tid, const pthread_attr_t *attr,  
                    void * ( *start_func) (void *), void *arg );
```

- se la chiamata ha successo, *tid* punta al thread ID;
- *attr* permette di specificare gli attributi del thread (se *attr* = NULL, gli attributi sono quelli di default);
- *start_func* è l'indirizzo della funzione di avvio;
- *arg* è l'indirizzo dell'argomento accettato dalla funzione di avvio;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di <sys/errno.h> – in caso di errore.

Pthreads

```
typedef void (*thread_start)(void *);
```

```
int pthread_create(pthread_t      *tid,  
                  const pthread_attr_t *attributes  
                  thread_start    start,  
                  void             *argument);
```

- Restituisce 0 se OK, un codice d'errore altrimenti
- tid = argomento di ritorno, conterrà il tid del nuovo thread
- attributes = attributi del thread (vedere dopo)
- start = indirizzo della funzione da cui partire
- argument = l'argomento passato alla funzione start

Pthreads

```
void pthread_exit(void *status);
```

- termina il thread corrente, con valore di uscita status
- altri thread possono raccogliere il valore di uscita usando `pthread_join` (vedere slide successiva)
- fare attenzione che i dati puntati da ret sopravvivano alla terminazione del thread!
 - status non deve puntare allo stack (no variabili locali)
 - Ok uso di variabili globali o allocate dinamicamente

Pthreads

Un thread può attendere per la terminazione di un altro thread relativo allo stesso processo:

```
#include <pthread.h>
```

```
int pthread_join ( pthread_t *tid, void **status );
```

- *tid* è l'ID del thread del quale si vuole attendere la terminazione;
- *status* punta al valore restituito dal thread per cui si è atteso, indicante il suo stato di terminazione (se *status* = NULL, tale stato non viene restituito);
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di `<sys/errno.h>` – in caso di errore.

Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```

Pthreads

```
/* thread_create: stampa i TID del main thread e di due altri
thread */

#include <pthread.h>
#include <stdio.h>
#include <errno.h>

void *start_func(void *arg) /* funzione di avvio */
{
    printf("%s", (char *)arg);
    printf(" and my TID is: %d\n", (int)pthread_self());
}

int main(void)
{
    int en;
    pthread_t tid1, tid2;
    char *msg1 = "Hello world, I am thread #1";
    char *msg2 = "Hello world, I am thread #2";

    printf("The launching process has PID:%d\n", (int)getpid());

    printf("The main thread has TID:%d\n", (int)pthread_self());
```

Pthreads

```
/* crea il 1mo thread */
if ((en = pthread_create(&tid1, NULL, start_func, msg1) != 0))
    errno=en, perror("pthread_create"), exit(1);

/* crea il 2ndo thread */
if ((en = pthread_create(&tid2, NULL, start_func, msg2) != 0))
    errno=en, perror("pthread_create"), exit(2);

/* attende per il 1mo */
if ((en = pthread_join(tid1, NULL) != 0))
    errno=en, perror("pthread_join"), exit(1);

/* attende per il 2ndo */
if ((en = pthread_join(tid2, NULL) != 0))
    errno=en, perror("pthread_join"), exit(2);

return 0;
}
```

Pthreads

```
typedef struct foo{  
    int a;  
    int b;  
} myfoo;
```

```
myfoo test; // Variabile GLOBALE
```

```
void stampa(char *st, struct foo *test){  
    printf("%s: tid=%d a=%d b=%d\n", st, pthread_self(), test->a, test->b);  
}
```

```
void *fun1(void *arg){  
    myfoo test2 = {1,2}; // Variabile LOCALE  
    printf("%s %d\n", arg, pthread_self());  
    stampa(arg, &test2);  
    pthread_exit((void *)&test2);  
}
```

Pthreads

```
void *fun2(void *arg){
    test.a = 3;
    test.b = 4; // Variabile GLOBALE
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, &test);
    pthread_exit((void *)&test);
}
```

```
void *fun3(void *arg){
    myfoo *test3;
    test3=malloc(sizeof(struct foo)); // Variabile allocata dinamicamente
    test3->a = 5;
    test3->b = 6;
    printf("%s %d\n", arg, pthread_self());
    stampa(arg, test3);
    pthread_exit((void *)test3); //c
}
```

Pthreads

```
int main(void){
    char st[100];
    pthread_t tid1;
    pthread_t tid2;
    pthread_t tid3;

    myfoo *b; // PUNTATORE alla struttura (non allocata)

    pthread_create(&tid1, NULL, fun1, "Thread 1"); // Locale
    pthread_join(tid1, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid2, NULL, fun2, "Thread 2"); // Globale
    pthread_join(tid2, (void *)&b);
    stampa("Master ", b);

    pthread_create(&tid3, NULL, fun3, "Thread 3"); // Dinamica
    pthread_join(tid3, (void *)&b);
    stampa("Master ", b);
}
```

Pthreads

Thread 1: 1077283760

// Locale

Thread 1: a=1 b=2

Master : a=1075156600 b=1077281896

Thread 2: 1077283760

// Globale

Thread 2: a=3 b=4

Master : a=3 b=4

Thread 3: 1077283760

// Dinamica

Thread 3: a=5 b=6

Master : a=5 b=6

Windows Multithreaded in C

<pre>#include <windows.h> #include <stdio.h> DWORD Sum; /* data is shared by the thread(s) */ /* the thread runs in this separate function */ DWORD WINAPI Summation(LPVOID Param) { DWORD Upper = *(DWORD*)Param; for (DWORD i = 0; i <= Upper; i++) Sum += i; return 0; }</pre>	Windows API
<pre>int main(int argc, char *argv[]) { DWORD ThreadId; HANDLE ThreadHandle; int Param; if (argc != 2) { fprintf(stderr, "An integer parameter is required\n"); return -1; } Param = atoi(argv[1]); if (Param < 0) { fprintf(stderr, "An integer >= 0 is required\n"); return -1; } }</pre>	DWORD 32-bit unsigned int
	LPVOID puntatore void
	Funzione di avvio del thread come in Pthread

Windows Multithreaded in C

```
/* create the thread */
ThreadHandle = CreateThread(
    NULL, /* default security attributes */
    0, /* default stack size */
    Summation, /* thread function */
    &Param, /* parameter to thread function */
    0, /* default creation flags */
    &ThreadId); /* returns the thread identifier */

if (ThreadHandle != NULL) {
    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
}
```

Creazione del thread come in Pthread

Attesa del thread come in Pthread

Thread in Java

- thread in Java gestiti dalla JVM
- Tipicamente implementati con il modello di thread fornito dal sistema operativo utilizzato
 - Esempio in Unix Pthreads
- I thread Java possono essere creati in due modi:
 - Estendendo la classe Thread con override del metodo run()

```
public class ThreadA extends Thread{  
    @Override  
    public void run(){  
        System.out.println("Codice del thread A");  
    }  
}
```

- Implementando l'interfaccia Runnable

```
public interface Runnable  
{  
    public abstract void run();  
}
```

Thread in Java

- ❑ thread in Java gestiti dalla JVM
- ❑ Tipicamente implementati con il modello di thread fornito dal sistema operativo utilizzato
 - ❑ Esempio in Unix Pthreads
- ❑ I thread Java possono essere creati in due modi:
 - ❑ Estendendo la classe Thread con override del metodo run()

```
public class Thread1 extends Thread{  
    @Override  
    public void run(){  
        System.out.println("Codice del thread 1");  
    }  
}
```

```
public class ProvaThread1 {  
    public static void main(String[] args) {  
        Thread1 thread1 = new Thread1();  
        thread1.start();  
    }  
}
```

thread1.start(); rende il thread eleggibile per l'esecuzione.

Una volta in esecuzione parte il run()

Thread in Java

- ❑ thread in Java gestiti dalla JVM
- ❑ Tipicamente implementati con il modello di thread fornito dal sistema operativo utilizzato
 - ❑ Esempio in Unix Pthreads
- ❑ I thread Java possono essere creati in due modi:
 - ❑ Estendendo la classe Thread con override del metodo run()
 - ❑ Implementando l'interfaccia Runnable

```
public interface Runnable
{
    public abstract void run();
}
```

```
public class Thread2 implements Runnable{
    public void run(){
        System.out.println("Codice del thread 2");
    }
}
```

Thread in Java

- thread in Java gestiti dalla JVM
- Tipicamente implementati con il modello di thread fornito dal sistema operativo utilizzato
 - Esempio in Unix Pthreads
- I thread Java possono essere creati in due modi:
 - Estendendo la classe Thread con override del metodo run()
 - Implementando l'interfaccia Runnable

```
public class Thread2 implements Runnable{
    public void run(){
        System.out.println("Codice del thread 2");
    }
}

public class ProvaThread2 {
    public static void main(String[] args) {
        Thread thread = new Thread(new Thread2());
        thread.start();
    }
}
```

Java Multithreaded Program

```
class Sum
{
    private int sum;

    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable
{
    private int upper;
    private Sum sumValue;

    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}
```

Summation implementa
l'interfaccia Runnable

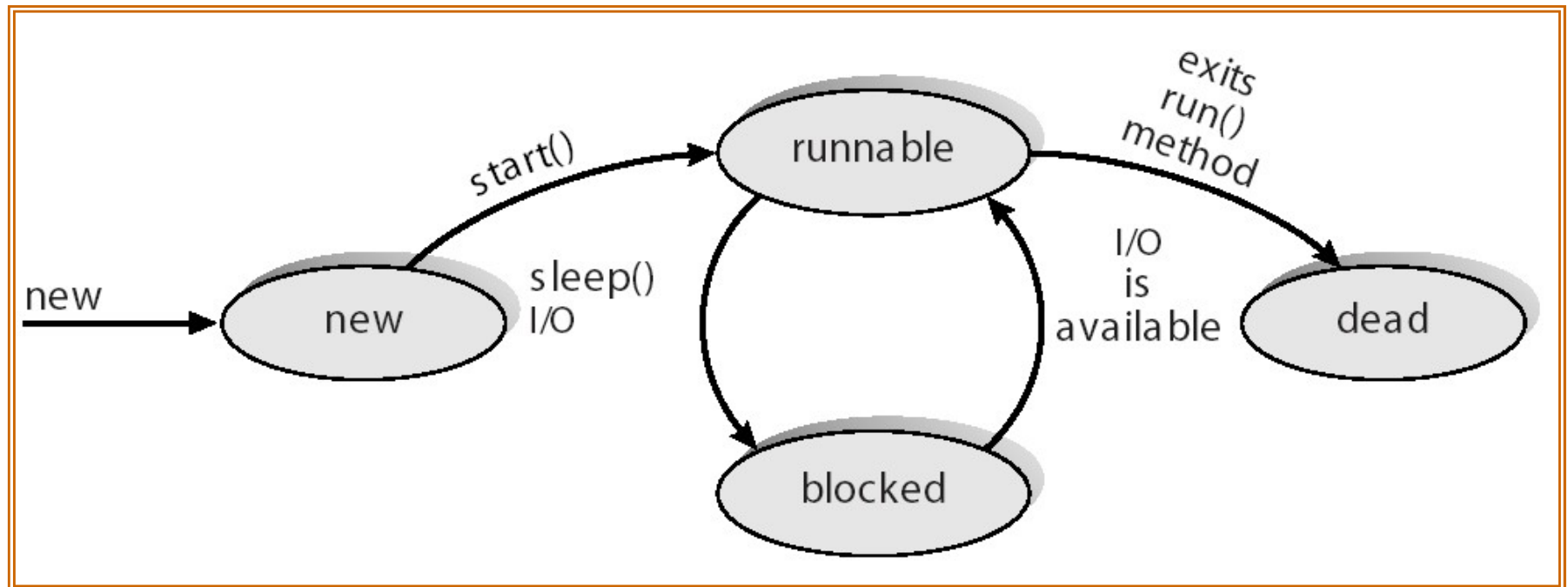
Il metodo run implementa
il thread

Java Multithreaded Program

```
public class Driver
{
    public static void main(String[] args) {
        if (args.length > 0) {
            if (Integer.parseInt(args[0]) < 0)
                System.err.println(args[0] + " must be >= 0.");
            else {
                Sum sumObject = new Sum();
                int upper = Integer.parseInt(args[0]);
                Thread thrd = new Thread(new Summation(upper, sumObject));
                thrd.start();
                try {
                    thrd.join();
                    System.out.println
                        ("The sum of "+upper+" is "+sumObject.getSum());
                } catch (InterruptedException ie) { }
            }
        }
        else
            System.err.println("Usage: Summation <integer value>"); }
}
```

Creazione del thread
Lancio con start
Raccolto con join

Java Multithreaded Program



Threading implicito

- Tecnica sempre più popolare al crescere della complessità dei sistemi multithread
- Creazione e gestione dei thread fatta da compilatori e librerie run-time invece che dai programmatori
- Il programmatore identifica dei **task** non dei thread

- Tre metodi considerati
 - Thread Pools
 - OpenMP
 - Grand Central Dispatch

Pool di Thread

- Si crea un numero di thread in attesa in un pool
 - Richiesto un thread per servizio, se non c'è attesa.
- Vantaggio:
 - Solitamente più veloce nel servire una richiesta quando il thread esiste già
 - Il numero dei thread per l'applicazione è limitato dalla dimensione del pool
 - Separa il task da eseguire dal meccanismo per creare il task e permette differenti strategie di esecuzione del task
 - ▶ i.e. I task possono essere schedulati per avviarsi periodicamente
- Windows API per supportare i thread pool:
 - Thread function da eseguire con un thread del pool

```
DWORD WINAPI PoolFunction( PVOID Param) {  
    /*  
    * this function runs as a separate thread.  
    */  
}
```

Pool di Thread

```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /* This function runs as a separate thread */  
}
```

- Un puntatore a `PoolFunction()` è passato ad una delle funzioni delle API del thread pool e un thread dal pool la esegue.
- Una di queste funzioni è `QueueUserWorkItem` che ha 3 parametri
 - `LPTHREAD_START_ROUTINE Function` – il puntatore alla funzione da eseguire come thread
 - `PVOID Param` – i parametri passati alla funzione
 - `ULONG Flags` – flag che indicano come il thread pool deve creare e gestire l'esecuzione del thread.

Esempio di chiamata:

```
QueueUserWorkItem(&PoolFunction, NULL, 0);
```

che porta uno dei thread del pool ad invocare la `PoolFunction()`

OpenMP

- Direttive di compilazione ed API per C, C++, FORTRAN
- Supporto per programmazione parallela in ambienti in shared-memory
- Identifica **regioni parallele** – blocchi di codice parallelizzabili

#pragma omp parallel

Crea tanti thread quanti sono i core

Esegui il loop in parallelo

```
#pragma omp parallel for
  for(i=0;i<N;i++) {
    c[i] = a[i] + b[i];
  }
```

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```

Problematiche Threading

- Semantica delle chiamate **fork()** ed **exec()**
- Signal handling
 - Sincrono e asincrono
- Cancellazione thread
 - Asincrono o deferred
- Thread-local storage
- Attivazione Scheduler

Semantica di `fork()` ed `exec()`

- **`fork()`** duplica solo il thread chiamante o tutti i thread del processo?
 - Per alcune versioni di UNIX due versioni di `fork`
 - Spesso duplica solo il thread chiamante

- **`exec()`** funziona normalmente – sovrascrive il processo in esecuzione inclusi i thread

Signal Handling

- Un “segnale” e' un interrupt “software”
 - La terminologia corretta e' “exception” mentre “interrupt” e' usata solo per gli interrupt “hardware”
- Consente la comunicazione asincrona tra processi e/o tra device e processo
- Ogni segnale ha un proprio nome
 - Tutti i nomi cominciano per “SIG”
 - Definiti in <signal.h>
 - Associati ad interi positivi

Signal Handling

Segnali in UNIX notificano eventi ai processi

- ❓ Un **signal handler** usato per processare i segnali
 1. Generato da evento
 2. Mandato ad un processo
 3. Signal handler:
 1. default
 2. user-defined

- ❓ Ogni segnale ha un **default handler** che il kernel esegue
 - ❓ **User-defined signal handler** sovrascrive il default
 - ❓ Per single-threaded, segnali mandati al processo

Segnali (Unix)

- **Caratteristiche dei segnali**
 - Ogni segnale ha un identificatore
 - Identificatori di segnali iniziano con i tre caratteri SIG
 - Es. **SIGABRT** è il segnale di abort
 - Numero segnali: 15-40, a seconda della versione di UNIX
 - POSIX: 18
 - Linux: 38
 - I nomi simbolici corrispondono ad un intero positivo
 - Definizioni di costanti in **bits/signal.h**

Segnali (Unix)

Pressione di tasti speciali sul terminale

- Es: Premere il tasto `ctrl-c` genera il segnale `SIGINT`

Eccezioni hardware

- Divisione per 0 (`SIGFPE`)
- Riferimento non valido a memoria (`SIGSEGV`)
- L'interrupt viene generato dall'hardware, e catturato dal kernel; questi invia il segnale al processo in esecuzione

System call `kill`

- Permette di spedire un segnale ad un altro processo
- Limitazione: uid del processo che esegue `kill` deve essere lo stesso del processo a cui si spedisce il segnale, oppure 0 (root)

Segnali (Unix)

<i>nome</i>	<i>significato</i>	<i>default</i>
SIGINT	interruzione da tastiera (Ctrl-c)	terminare
SIGSTOP*	stop al processo	<i>fermare</i>
SIGKILL*	terminazione forzata	terminare
SIGQUIT	quit da tastiera (Ctrl-y)	terminare
SIGTERM	terminazione da tastiera (Ctrl-\\)	terminare
SIGCHLD	figlio terminato o fermato	<i>ignorare</i>
SIGALRM	suona la sveglia!	terminare
SIGSEGV	segmentation fault	terminare
SIGUSR1	a disposizione dell'utente	terminare

Segnali (Unix)

- I segnali vengono inviati in modo asincrono.
 - Non e' possibile sapere quando il processo riceverà un segnale.
- E' possibile indicare al kernel l'azione da intraprendere quando un segnale e' generato per un processo:
 - Ignora: Valida per quasi tutti i segnali tranne SIGKILL e SIGSTOP.
 - “Catch” del segnale: Indicare una procedura da eseguire (signal handler). Ad esempio:
 - SIGCHLD: esegui le operazioni associate alla terminazione di un figlio
 - SIGINT: (CTRL-C) “cancella file temporanei”...
 - **Non e' possibile intercettare SIGKILL o SIGSTOP.**
 - Default: Eseguire l'azione di default.

Segnali (Unix)

- Un handler (gestore) è una funzione del tipo:

```
void funzione(int num_segnaile) {  
    printf("%d", num_segnaile);  
}
```

Una volta che l'handler termina, si torna al punto in cui il programma era stato interrotto.

Segnali (Unix)

```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

- `signal(SIGINT, foo)` imposta la funzione `foo` come handler del segnale `SIGINT`
- si può anche richiedere di ignorare il segnale
 - `signal(SIGINT, SIG_IGN)`
- oppure ritornare alla reazione di default
 - `signal(SIGINT, SIG_DFL)`

Segnali (Unix)

int **kill**(pid_t pid, int sig);

- kill(127, SIGINT) invia il segnale SIGINT al processo il cui pid e' 127
- restituisce 0 in caso di successo e -1 in caso di errore

Segnali (Unix)

```
void foo(int num_segnaile);
int main(void){

    int n=0;  int buf[100];
    alarm(5);
    signal(SIGALRM,foo);

    while (n<=0){
        printf("Digitare qualcosa:\n");
        alarm(1);
        if ((n=read(STDIN_FILENO,buf,10))<0)
            perror("Read error");
        alarm(0);
    }
}

void foo(int num_segnaile) {
    alarm(1);
    printf("Vuoi muoverti a digitare qualcosa ???\n");
}
```


Signal Handling

Se multi-threaded?

- ❑ Invia il segnale al thread a cui si applica
- ❑ Invia a tutti i thread del processo
- ❑ Invia a specifici thread del processo
- ❑ Assegna ad un thread specifico il compito di ricevere tutti i segnali del processo

Signal Handling

```
int pthread_kill(pthread_t tid, int signo);
```

- manda il segnale signo al thread specificato da tid
 - se e' impostato un handler, viene eseguito nel thread tid
 - se non e' impostato un handler, e il comportamento di default e' di terminare il processo, vengono comunque terminati tutti i thread
- restituisce 0 se OK, un codice d'errore altrimenti

Signal Handling

```
signal(SIGUSR1, usr1);
pthread_create(&tid1, NULL, fun, "Thread 1");
pthread_create(&tid2, NULL, fun, "Thread 2");
pthread_create(&tid3, NULL, fun, "Thread 3");
sleep(1);
pthread_kill(tid1, SIGUSR1);
pthread_kill(tid2, SIGUSR1);
pthread_kill(tid3, SIGUSR1);
```

(USR1 = User defined signal)

```
sigemptyset(&set); // Configura la maschera SOLO nel master thread
sigaddset(&set, SIGUSR1);
sigprocmask(SIG_SETMASK, &set, NULL);
sleep(1);
while (i++<10){
    sleep(1);
    kill(pid, SIGUSR1); // il segnale e' intercettato da un thread
}
```

Dove le maschere dei segnali sono sia al livello di processi che thread:

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

```
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

Signal Handling

Thread id=1077283760 ricevuto segnale

Thread id=1079385008 ricevuto segnale

Thread id=1081486256 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Thread id=1077283760 ricevuto segnale

Cancellazione Thread

- ❑ Terminare un thread prima che sia finito
- ❑ Il thread da cancellare è il **target thread**
- ❑ Due approcci:
 - ❑ **Asynchronous cancellation** termina il target thread subito
 - ❑ **Deferred cancellation** permette al target thread di controllare periodicamente se deve essere cancellato
- ❑ Codice Pthread per creare e cancellare un thread:

```
pthread_t tid;  
  
/* create the thread */  
pthread_create(&tid, 0, worker, NULL);  
  
. . .  
  
/* cancel the thread */  
pthread_cancel(tid);
```

Cancellazione Thread

- La cancellazione effettiva dipende dallo stato del thread

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- Se il thread ha la cancellazione disabilitata questa rimane pending finché il thread non la consente
- Il default è deferred:
 - Cancellazione avviene solo quando il thread raggiunge un **cancellation point**
 - ▶ Allora **cleanup handler** è invocato
- Su Linux la cancellazione dei thread gestita con segnali

Cancellazione Thread (Posix)

- In ogni istante, un thread può essere cancellabile o non cancellabile
- Quando partono tutti i thread sono cancellabili
- Quando un altro thread chiama `pthread_cancel`
 - se il thread è cancellabile, viene cancellato
 - se non è cancellabile, la richiesta di cancellazione viene memorizzata, in attesa che il thread diventi cancellabile

Cancellazione Thread (Posix)

```
int pthread_setcancelstate(int state, int *oldstate);
```

- imposta la cancellabilità a state e restituisce la vecchia cancellabilità in oldstate
- state e oldstate possono assumere i valori:
 - PTHREAD_CANCEL_ENABLE
 - PTHREAD_CANCEL_DISABLE
- restituisce 0 se OK, un codice d'errore altrimenti

Thread-Specific Data

- **Thread-local storage (TLS)** o **Thread-specific Data (TSD)**
 - Nei thread var globali condivise, var locali no
 - Ogni thread può mantenere una sua copia dei dati
 - Utile quando non si controlla direttamente la creazione dei thread (i.e., usando i thread pool)

- Differenza rispetto a variabili locali
 - Variabili locali visibili solo per una singola invocazione di funzione
 - TSD visibile per più invocazioni di funzione

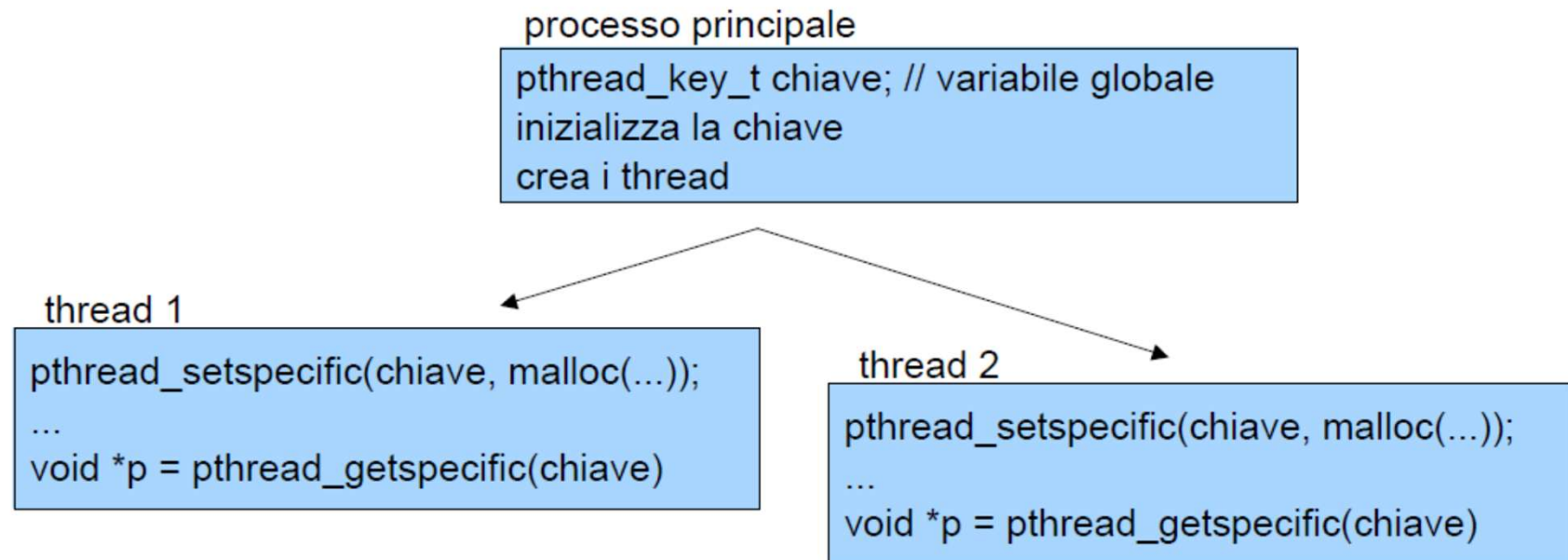
- Simile ai dati **static** (ma senza race condition)
 - TSD unico per ogni thread

Thread-Specific Data (Posix)

- Ogni thread possiede un'area di memoria privata, la TSD area, indicizzata da chiavi
- La TSD area contiene associazioni tra le chiavi ed un valore di tipo void*
 - diversi thread possono usare le stesse chiavi ma i valori associati variano di thread in thread
 - inizialmente tutte le chiavi sono associate a NULL

Thread-Specific Data (Posix)

- associare a una stessa chiave, dati diversi per ciascun thread



Thread-Specific Data (Posix)

- `int pthread_key_create(...)`
 - per creare una chiave TSD
- `int pthread_key_delete(...)`
 - per deallocare una chiave TSD
- `int pthread_setspecific(...)`
 - per associare un certo valore ad una chiave TSD
- `void * pthread_getspecific(...)`
 - per ottenere il valore associato ad una chiave TSD

Thread-Specific Data (Posix)

```
int pthread_key_create(pthread_key_t *key,  
                      void (*destructor)(void *));
```

- crea una chiave per dati privati
- key è l'indirizzo della chiave da inizializzare
- destructor è un puntatore alla funzione distruttore che deve essere chiamata alla terminazione di un thread (pthread_exit())
- restituisce 0 se OK, un codice d'errore altrimenti

Thread-Specific Data (Posix)

```
int pthread_setspecific(pthread_key_t *key,  
                        const void* val);
```

- associa l'indirizzo val alla chiave key, per il thread chiamante
 - restituisce 0 se OK, un codice d'errore altrimenti
-

```
void* pthread_getspecific(pthread_key_t *key);
```

- restituisce l'indirizzo associato alla chiave key nel thread chiamante
 - restituisce NULL se nessun indirizzo è stato associato a key

Thread-Specific Data (Posix)

```
#include ...

static pthread_key_t thread_log_key; /* tsd key per thread */

void write_to_thread_log (const char* message); //Scrive log
void close_thread_log (void* thread_log); //Chiude file log
void* thread_function (void* args);          //Eseguita dai thread

int main() {
    // Crea una chiave da associare al log Thread-Specific
    // Crea 5 thread che facciano il lavoro
    // Aspetta che tutti finiscano
    return 0;
}

...
```

Thread-Specific Data (Posix)

```
...
int main() {
    int i;
    pthread_t threads[5];
    // Crea una chiave da associare al puntatore TSD al log file
    pthread_key_create(&thread_log_key, close_thread_log);

    for (i = 0; i < 5; ++i) // thread che faccia il lavoro
        pthread_create(&(threads[i]), NULL, thread_function, NULL);

    for (i = 0; i < 5; ++i) // Aspetta che tutti finiscano
        pthread_join (threads[i], NULL);
    return 0;
}
```


Thread-Specific Data (Posix)

```
...
void write_to_thread_log (const char* message) {
    FILE* thread_log = (FILE*)pthread_getspecific(thread_log_key);
    fprintf (thread_log, "%s\n", message);
}

void close_thread_log (void* thread_log) {
    fclose ((FILE*) thread_log);
}

void* thread_function (void* args) {
    char thread_log_filename[20];
    FILE* thread_log;
    sprintf(thread_log_filename, "thread%d.log", (int)pthread_self ());

    thread_log = fopen (thread_log_filename, "w");
    /* Associa la struttura FILE TSD a thread_log_key. */
    pthread_setspecific (thread_log_key, thread_log);

    write_to_thread_log ("Thread starting.");
    /* Fai altro lavoro qui... */ return NULL;
}
```

Esempi in SO

- Linux Threads
- Windows Threads

Thread Linux

- ❑ Linux si riferisce a **tasks** piuttosto che a **threads**
 - ❑ Unificando la gestione di processi e thread
- ❑ Thread creation è gestita dalla chiamata di sistema **clone()**
 - ❑ Usata per implementare **pthread_create**
- ❑ **clone()** permette ad un child task di condividere l'address space del parent task (processo)
 - ❑ Flag di controllo:

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

Viene comunque usata la chiamata a `fork()`:
`do_fork()`

- ❑ Se tutte settate allora clone come thread, se tutte non settate come fork
- ❑ **struct task_struct** punta alle strutture dati del processo (condivise o uniche) - gestione uniforme

Thread Linux

□ Esempio di uso del `clone()`

```
int thread_func(void *arg) {
    printf("Hello from the new thread! Arg: %s\n", (char *)arg);
    return 0;
}

int main() {
    char *child_stack = malloc(1024 * 1024); // Allocazione dello stack
    if (child_stack == NULL) {
        perror("malloc");
        exit(1);
    }

    // Creazione del nuovo thread con clone()
    pid_t pid = clone(thread_func, child_stack + 1024 * 1024, CLONE_VM | CLONE_FS | CLONE_FILES |
        CLONE_SIGHAND | CLONE_THREAD, "Thread Arg");

    if (pid == -1) {
        perror("clone");
        exit(1);
    }

    printf("Thread created with pid: %d\n", pid);
    sleep(1); // Aspetta che il thread finisca
    free(child_stack); // Libera lo stack

    return 0;
}
```

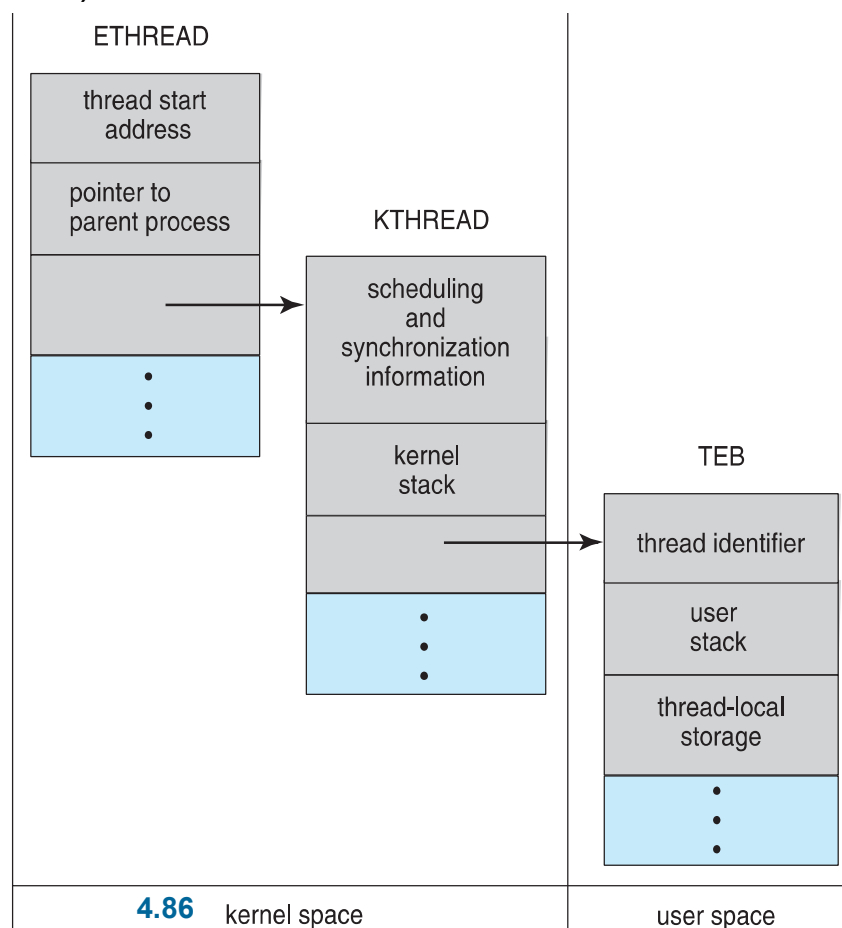
Windows Threads

- Processi possono contenere più thread
- Implementa il mapping one-to-one, kernel-level
- Ogni thread contiene
 - Un thread id
 - Program counter
 - Register set rappresenta lo stato del processore
 - User e kernel stacks separati per i thread eseguiti in user mode o kernel mode
 - Area di private data storage usati da run-time libraries e dynamic link libraries (DLLs)
- Register set, stacks, e private storage sono detti il **context** del thread

Windows Threads

□ Strutture dati del thread includono:

- ETHREAD (executive thread block) – include punt. al processo al quale il thread appartiene, punt. alla routine da dove parte il thread, punt. a KTHREAD nel kernel space
- KTHREAD (kernel thread block) – info di scheduling e sincronizzazione, kernel-mode stack, puntatore a TEB nel user space
- TEB (thread environment block) – thread id, user-mode stack, thread-local storage, nello user space



Windows Threads Data Structures

